指示編集ダイアログ UI仕様書

概要

目的

単一指示の新規作成・編集・詳細表示を行うモーダルダイアログコンポーネント。指示対象（プロジェクト全体/メソッド/タスク/Transition）の選択、優先度・タイプ・内容の入力、および連続指示機能を提供する。

使用場面

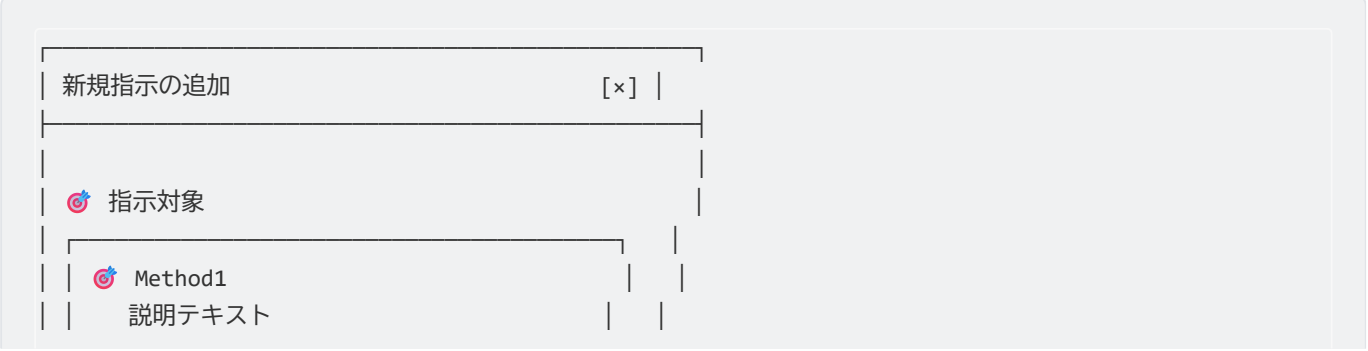
- **タスク登録タブ**: 各タスクに対する指示を追加（`fixedMethodId`指定）
- **指示登録タブ**: 指示の新規作成・編集・詳細表示
- **プロジェクト指示**: プロジェクト全体に対する指示を追加（`fixedMethodId=""`）

アクセス権限

- **Experiment Admin（実験管理者）**: 指示の作成・編集・閲覧
- **その他のロール**: アクセス不可

画面構成

レイアウト構造



```
│ └──────────────────────────────────────┘   │
│                                            │
│ 優先度: *                                  │
│ [高 ▼]                                     │
│                                            │
│ タイプ: *                                  │
│ [一般 ▼]                                   │
│                                            │
│ 🎯 対象タスク:                             │
│ [タスク番号を選択... ▼]                    │
│                                            │
│ 🔗 トランジションID:                       │
│ [トランジションを選択... ▼]                │
│                                            │
│ 指示内容: *                                │
│ ┌──────────────────────────────────────┐   │
│ │                                      │   │
│ │ [作業者への具体的な指示...]          │   │
│ │                                      │   │
│ │                              0/1000  │   │
│ └──────────────────────────────────────┘   │
│                                            │
│              [キャンセル] [連続指示] [保存] │
└────────────────────────────────────────────┘

【送信済み指示の詳細表示】
┌────────────────────────────────────────────┐
│ 指示詳細                              [×] │
├────────────────────────────────────────────┤
│ （全フィールドが非活性・表示のみ）         │
│                                [閉じる]    │
└────────────────────────────────────────────┘
```

UI要素

1. ダイアログヘッダー

- **新規作成**: 「新規指示の追加」
- **編集**: 「指示の編集」
- **送信済み詳細**: 「指示詳細」

2. 指示対象表示（読み取り専用）

- **背景色**: `token.colorPrimaryBg`（淡い青）

- **プロジェクト全体**: 📄 アイコン + 「プロジェクト全体」
- **メソッド指定**: 🎯 アイコン + メソッド名 + 説明

3. 入力フィールド

フィールド	必須	タイプ	説明
優先度	✓	Select	低/通常/中/高（コードマスタから取得）
タイプ	✓	Select	一般/セットアップ/測定/安全/清掃
対象タスク	-	Select	タスク番号（1〜trialCount）
トランジションID	-	Select	メソッド内のTransition一覧
指示内容	✓	TextArea	1000文字以内、6行、文字数カウント表示

4. フッターボタン

未送信指示の場合:

- **キャンセルボタン**: ダイアログを閉じる（変更破棄）
- **連続指示ボタン**: 保存後もダイアログを開いたまま（新規作成時のみ）
- **保存ボタン**: 指示を保存してダイアログを閉じる

送信済み指示の場合:

- **閉じるボタン**: ダイアログを閉じる（編集・削除不可）

機能仕様

Props定義

```
interface InstructionEditDialogProps {
  open: boolean;                        // ダイアログ表示状態
  mode: 'create' | 'edit';              // モード（作成/編集）
  instruction?: InstructionData;        // 編集対象の指示
  tasks: TaskDetail[];                  // タスク一覧
  availableMethods?: MethodInfo[];      // メソッド一覧
  availableTransitions: TransitionInfo[]; // Transition一覧（全メソッド分）
```

```
  onClose: () => void;                    // 閉じるハンドラー
  onSave: (instruction: InstructionData, keepDialogOpen?: boolean) => void; // 保存ハンドラー
  fixedMethodId?: string;                 // 固定するメソッドID（指定時はメソッド選択を無効化）
  saveMode?: 'immediate' | 'local';       // 保存モード
}
```

fixedMethodId:

- 指定あり: メソッドを固定（タスク登録タブから使用）
- `""`: プロジェクト全体に対する指示
- 未指定: メソッド選択可能（指示登録タブから使用）

saveMode:

- `'immediate'`: 即時DB保存（指示登録タブのデフォルト動作）
- `'local'`: ローカルstateに保存（タスク登録タブで使用）

State管理

```
// フォーム状態
const [form] = Form.useForm();

// 選択状態
const [selectedMethodId, setSelectedMethodId] = useState<string>('');
const [selectedTaskNumber, setSelectedTaskNumber] = useState<number | undefined>(undefined);
const [selectedTransitionId, setSelectedTransitionId] = useState<string>('');

// コードマスタ
const [priorityOptions, setPriorityOptions] = useState<SelectOption[]>([]);
const [typeOptions, setTypeOptions] = useState<SelectOption[]>([]);
const [isLoadingCodeMaster, setIsLoadingCodeMaster] = useState<boolean>(false);
```

Context連携

```
const { maimlRoot } = useProjectMaiml(); // MAIMLデータ取得（トランジションのフィルタリング用）
```

主要機能

1. コードマスタの取得

実行タイミング: コンポーネントマウント時

```
useEffect(() => {
  const fetchCodeMasterOptions = async () => {
    try {
      setIsLoadingCodeMaster(true);
      const options = await getMultipleSelectOptions([
        'instruction_priority',
        'instruction_type'
      ]);

      setPriorityOptions(options.instruction_priority || []);
      setTypeOptions(options.instruction_type || []);
    } catch (error) {
      console.error('コードマスタ取得エラー:', error);
      // フォールバック: 空配列（ハードコードされた値を使用）
      setPriorityOptions([]);
      setTypeOptions([]);
    } finally {
      setIsLoadingCodeMaster(false);
    }
  };

  fetchCodeMasterOptions();
}, []);
```

API: `GET /api/v1/code-master/multiple?keys=instruction_priority,instruction_type`

フォールバック:

- コードマスタ取得失敗時はハードコードされた値を使用
- 優先度: 低(0), 通常(1), 中(2), 高(3)
- タイプ: 一般(0), セットアップ(1), 測定(2), 安全(3), 清掃(4)

2. トランジションの動的フィルタリング

実行タイミング: `selectedMethodId` が変更された時

```
const filteredTransitions = React.useMemo((): TransitionInfo[] => {
  if (!selectedMethodId || !maimlRoot?.protocol?.methods) {
    return [];
  }

  const method = maimlRoot.protocol.methods.find(m => m.id === selectedMethodId);
  if (!method?.pnml?.transitions) {
    return [];
  }

  return method.pnml.transitions.map((transition) => ({
    id: transition.id,
    displayName: transition.name || transition.id,
    category: 'other',
    description: transition.description,
    isExperimentRelevant: true,
  }));
}, [selectedMethodId, maimlRoot]);
```

フィルタリングロジック:

- `selectedMethodId` が指定されている場合: そのメソッドのトランジションのみを表示
- `selectedMethodId` が空の場合: トランジション選択不可

3. ダイアログの初期化

実行タイミング: ダイアログが開かれた時（`open = true`）

編集モード:

```
if (mode === 'edit' && instruction) {
  form.setFieldsValue({
    priority: instruction.priority,
    type: instruction.type,
    methodId: instruction.methodId,
    taskNumber: instruction.taskNumber,
    transitionId: instruction.transitionId,
    content: instruction.content,
  });
  setSelectedMethodId(instruction.methodId);
  setSelectedTaskNumber(instruction.taskNumber);
```

```
  setSelectedTransitionId(instruction.transitionId);
}
```

新規作成モード:

```
const initialMethodId = fixedMethodId || '';
form.setFieldsValue({
  priority: INSTRUCTION_PRIORITIES_LOCAL.NORMAL, // 1
  type: INSTRUCTION_TYPES_LOCAL.GENERAL,         // 0
  methodId: initialMethodId,
  taskNumber: undefined,
  transitionId: '',
  content: '',
});
setSelectedMethodId(initialMethodId);
setSelectedTaskNumber(undefined);
setSelectedTransitionId('');
```

4. メソッド選択時の処理

```
const handleMethodChange = useCallback((value: string) => {
  setSelectedMethodId(value);
  // トランジションとタスク番号をリセット
  setSelectedTransitionId('');
  setSelectedTaskNumber(undefined);
  form.setFieldsValue({
    transitionId: '',
    taskNumber: undefined,
  });
}, [form]);
```

リセット理由: メソッドを変更すると、トランジション・タスク番号の選択肢が変わるため

5. タスク一覧の取得

```
const methodTasks = React.useMemo(() => {
  return tasks.filter(t => t.methodId === selectedMethodId);
}, [tasks, selectedMethodId]);
```

```
const selectedTask = methodTasks.length > 0 ? methodTasks[0] : undefined;
```

タスク番号の選択可否:

- `selectedMethodId` が空: 選択不可
- `selectedTask` が存在しない: 選択不可
- `selectedTask.experimentCondition !== 1` : 選択不可（実験条件のみ）
- 上記以外: 選択可能

6. 保存処理

共通処理:

```
const saveInstruction = useCallback(async (keepDialogOpen: boolean) => {
  try {
    const values = await form.validateFields();

    // バリデーション
    if (!values.content || values.content.trim() === '') {
      form.setFields([
        {
          name: 'content',
          errors: ['指示内容を入力してください'],
        },
      ]);
      return;
    }

    // 指示データを構築
    const instructionData: InstructionData = {
      id: instruction?.id || Date.now(),
      content: values.content.trim(),
      priority: values.priority,
      type: values.type,
      targetType: TARGET_TYPES_LOCAL.FILE, // デフォルト
      methodId: values.methodId || '',
      transitionId: values.transitionId || '',
      taskNumber: values.taskNumber,
    };

    // 対象タイプを判定
    if (instructionData.transitionId) {
      instructionData.targetType = TARGET_TYPES_LOCAL.TRANSITION; // 2
```

```
    } else if (instructionData.taskNumber !== undefined) {
      instructionData.targetType = TARGET_TYPES_LOCAL.TASK; // 1
    } else {
      instructionData.targetType = TARGET_TYPES_LOCAL.FILE; // 0
    }

    // 保存コールバック
    onSave(instructionData, keepDialogOpen);

    // ダイアログを開いたままにする場合のみフォームをリセット
    if (keepDialogOpen) {
      const currentMethodId = values.methodId || '';

      form.resetFields();
      form.setFieldsValue({
        priority: INSTRUCTION_PRIORITIES_LOCAL.NORMAL,
        type: INSTRUCTION_TYPES_LOCAL.GENERAL,
        methodId: currentMethodId, // メソッドIDを引き継ぐ
        taskNumber: undefined,
        transitionId: '',
        content: '',
      });
      // メソッドIDは保持、トランジションとタスク番号のみリセット
      setSelectedMethodId(currentMethodId);
      setSelectedTaskNumber(undefined);
      setSelectedTransitionId('');
    }
  } catch (error) {
    console.error('Validation failed:', error);
  }
}, [form, instruction, onSave]);
```

保存ボタン:

```
const handleSave = useCallback(async () => {
  await saveInstruction(false); // ダイアログを閉じる
}, [saveInstruction]);
```

連続指示ボタン:

```
const handleSaveAndContinue = useCallback(async () => {
  await saveInstruction(true); // ダイアログを開いたまま
}, [saveInstruction]);
```

対象タイプの判定優先順位:

1. `transitionId` が指定されている → `TARGET_TYPES_LOCAL.TRANSITION` (2)
2. `taskNumber` が指定されている → `TARGET_TYPES_LOCAL.TASK` (1)
3. 上記以外 → `TARGET_TYPES_LOCAL.FILE` (0)

7. キャンセル処理

```
const handleCancel = useCallback(() => {
  form.resetFields();
  setSelectedMethodId('');
  setSelectedTaskNumber(undefined);
  setSelectedTransitionId('');
  onClose();
}, [form, onClose]);
```

8. 送信済み指示の判定

```
const isSentInstruction = instruction?.isSent === true;
```

送信済み指示の場合:

- すべての入力フィールドが非活性（`disabled={isSentInstruction}`）
- フッターボタンは「閉じる」のみ
- ダイアログタイトルは「指示詳細」

データ型定義

InstructionData

```
interface InstructionData {
  id: string | number;                // 指示ID
```

```
  content: string;                    // 指示内容
  priority: number;                   // 優先度（0: 低, 1: 通常, 2: 中, 3: 高）
  type: number;                       // タイプ（0: 一般, 1: セットアップ, 2: 測定, 3: 安全, 4:
清掃）
  targetType: number;                 // 対象タイプ（0: ファイル, 1: タスク, 2: Transition）
  methodId: string;                   // メソッドID（空文字の場合はプロジェクト指示）
  transitionId: string;               // TransitionID
  taskNumber?: number;                // タスク番号
  isSent?: boolean;                   // 送信済みフラグ
}
```

定数

優先度（INSTRUCTION_PRIORITIES_LOCAL）:

```
const INSTRUCTION_PRIORITIES_LOCAL = {
  LOW: 0,      // 低
  NORMAL: 1,   // 通常
  MEDIUM: 2,   // 中
  HIGH: 3,     // 高
} as const;
```

タイプ（INSTRUCTION_TYPES_LOCAL）:

```
const INSTRUCTION_TYPES_LOCAL = {
  GENERAL: 0,     // 一般
  SETUP: 1,       // セットアップ
  MEASUREMENT: 2, // 測定
  SAFETY: 3,      // 安全
  CLEANUP: 4,     // 清掃
} as const;
```

対象タイプ（TARGET_TYPES_LOCAL）:

```
const TARGET_TYPES_LOCAL = {
  FILE: 0,        // ファイル（プロジェクト全体/メソッド）
  TASK: 1,        // タスク
  TRANSITION: 2,  // Transition
} as const;
```

バリデーション

必須項目

フィールド	バリデーション	エラーメッセージ
優先度	required	「優先度を選択してください」
タイプ	required	「タイプを選択してください」
指示内容	required	「指示内容を入力してください」
指示内容	max: 1000	「1000文字以内で入力してください」

カスタムバリデーション

指示内容の空白チェック:

```
if (!values.content || values.content.trim() === '') {
  form.setFields([
    {
      name: 'content',
      errors: ['指示内容を入力してください'],
    },
  ]);
  return;
}
```

特記事項

連続指示モード

機能概要:

- 保存後もダイアログを開いたまま、続けて次の指示を入力可能
- 新規作成モード時のみ「連続指示」ボタンを表示

動作:

1. 「連続指示」ボタンをクリック
2. 指示を保存（`onSave(instruction, true)`）
3. フォームをリセット
4. **メソッドIDは引き継ぐ**（トランジション・タスク番号のみリセット）
5. ダイアログは開いたまま

用途: 同じメソッドに対して複数の指示を連続登録する場合に便利

固定メソッドID（fixedMethodId）

指定パターン:

- `fixedMethodId={methodId}`: 特定メソッドに固定（タスク登録タブから使用）
- `fixedMethodId=""`: プロジェクト全体に固定（プロジェクト指示ボタンから使用）
- `fixedMethodId` 未指定: メソッド選択可能（指示登録タブから使用）

UI挙動:

- `fixedMethodId` が指定されている場合、メソッド選択フィールドは非表示（hiddenフィールドとして保持）
- 指示対象カードに固定されたメソッドまたは「プロジェクト全体」を表示

トランジション選択の制約

選択可能条件:

- `selectedMethodId` が指定されている
- MAIMLコンテキストから該当メソッドのトランジションを取得できる

表示内容:

- 実験関連のトランジションには⭐マーク
- カテゴリが'other'以外の場合は `[カテゴリ名]` を表示
- トランジション選択時に詳細情報をAlertで表示

タスク番号選択の制約

選択可能条件:

- `selectedMethodId` が指定されている
- 該当メソッドのタスクが存在する
- ** `experimentCondition === 1` **（実験条件のみ）

理由: テスト条件（0）のタスクには指示を追加できない

送信済み指示の扱い

判定: `instruction?.isSent === true`

UI変更:

- ダイアログタイトル:「指示詳細」
- すべての入力フィールドが非活性
- フッターボタン:「閉じる」のみ
- 編集・削除不可（閲覧のみ）

保存モード（saveMode）

immediate（デフォルト）:

- 指示登録タブから使用
- 保存時に即座にAPIを呼び出してDB保存
- 親コンポーネント（InstructionRegistrationTab）で実装

local:

- タスク登録タブから使用
- 保存時はローカルstateに追加のみ（API呼び出しなし）
- 「保存」ボタンクリック時に一括でAPI送信
- 親コンポーネント（TaskRegistrationTab）で実装

メソッド情報の取得

availableMethods（Props）:

- 指示登録タブから渡される（MAIMLコンテキストから生成）
- タスク登録タブでは未指定（タスク一覧から推測）

メソッド表示:

```
const selectedMethod = availableMethods?.find(m => m.id === selectedMethodId);
```

- `selectedMethod?.displayName`: メソッド名
- `selectedMethod?.description`: メソッドの説明

エラーハンドリング

コードマスタ取得失敗

対処:

- エラーをコンソールに出力
- フォールバック: ハードコードされた値を使用
- 優先度: 低(0), 通常(1), 中(2), 高(3)
- タイプ: 一般(0), セットアップ(1), 測定(2), 安全(3), 清掃(4)

フォームバリデーション失敗

対処:

- エラーをコンソールに出力
- フォームのエラー表示を更新
- ダイアログを閉じない

API連携

1. コードマスタ取得

エンドポイント: `GET /api/v1/code-master/multiple?keys=instruction_priority,instruction_type`

レスポンス:

```
{
  instruction_priority: Array<{
    value: number;
    label: string;
  }>;
  instruction_type: Array<{
    value: number;
    label: string;
  }>;
}
```

実装ファイル情報

パス

```
frontend/src/components/project/InstructionEditDialog.tsx
```

主要な依存パッケージ

- `antd` : UI コンポーネント（Modal, Form, Select, Input, Space, Typography, Alert, Card, theme, Button）
- `@ant-design/icons` : アイコン（FileTextOutlined, AimOutlined, PlusOutlined, SaveOutlined）
- `react` : React本体（useState, useEffect, useCallback）

Context

- `ProjectMaimlContext` : MAIMLデータ取得（トランジションのフィルタリング用）

API Service

- `codeMasterApiService` : コードマスタ取得

変更履歴

バージョン	日付	変更内容	担当者
1.0	2026-03-11	実装ベースで仕様書を新規作成（実装ファイル: InstructionEditDialog.tsx）	Cline

---

関連ドキュメント

- プロジェクト管理画面
- タスク登録タブ
- 指示登録タブ
- タスク結果一覧テーブル